

함수 1:

단일 행 함수

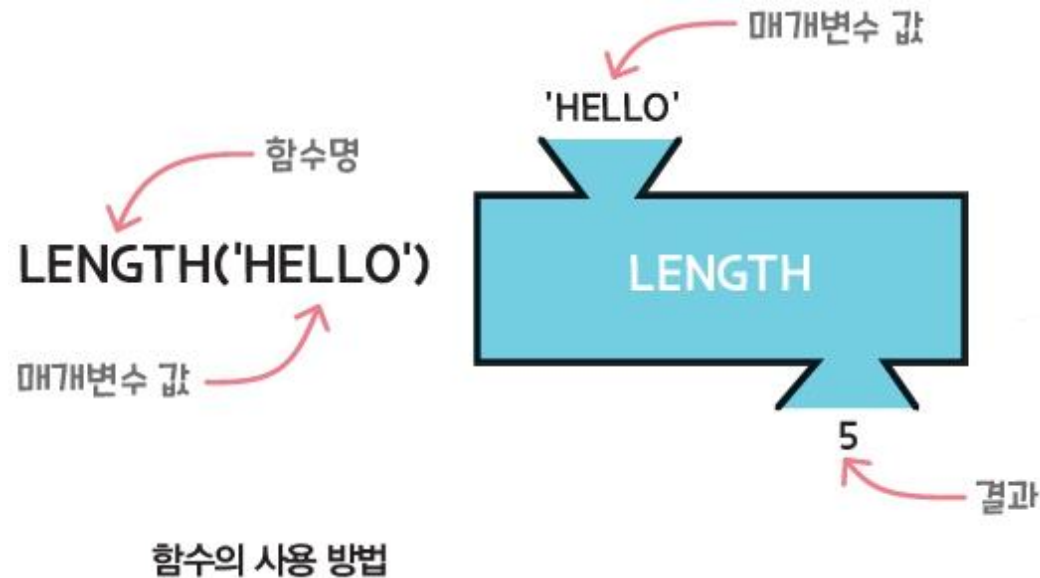
Section 01

SQL 함수의 개요

함수의 개념

● 함수

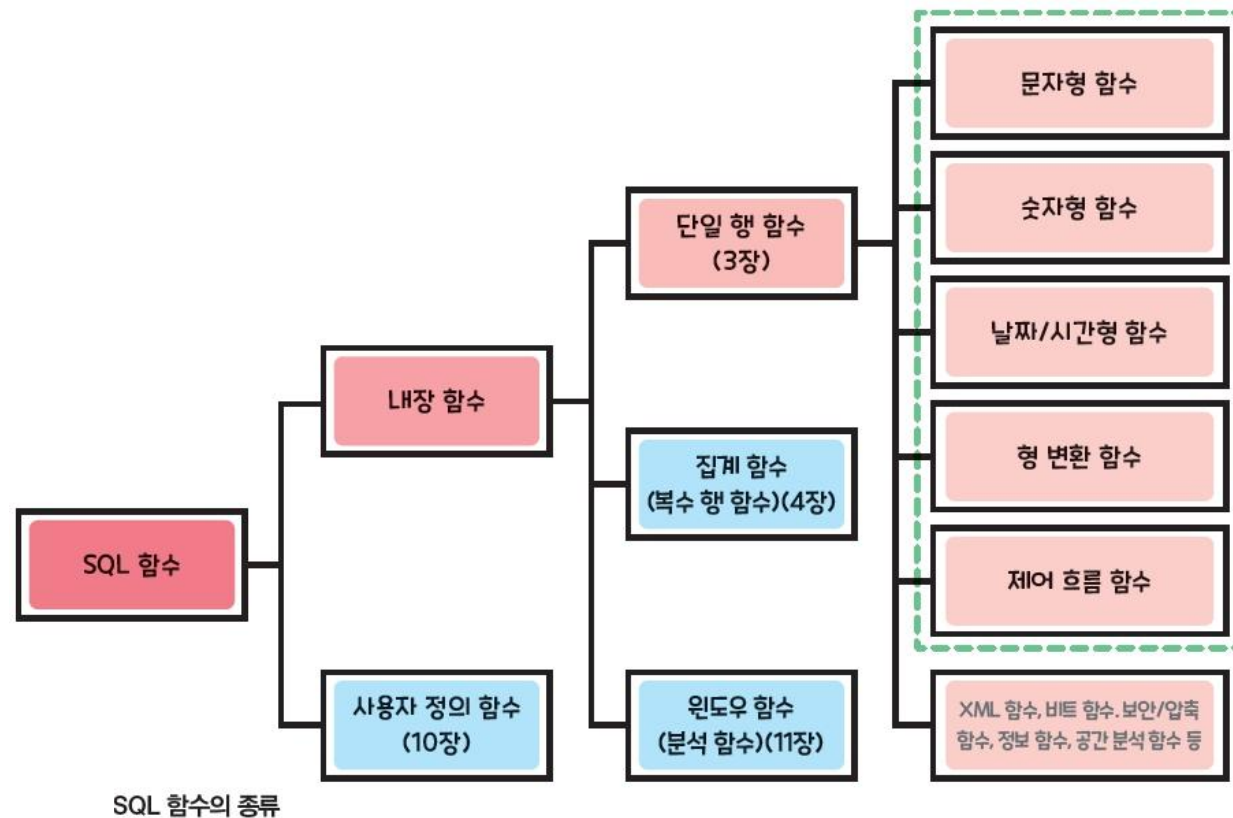
- 특별한 목적의 작업을 수행하기 위해 독립적으로 설계된 프로그램 코드의 집합
- 사용자는 어떤 로직에 의해서 함수가 동작하는지에 대해 알 필요 없이 필요한 값을 주고, 원하는 값을 얻으면 됨
- 함수를 실행할 때 필요한 값은 매개변수를 통해 전달됨
- [예] 문자열의 길이를 반환하는 LENGTH 함수
 - 함수 사용에 필요한 매개변수 값으로 문자열을 입력함



SQL 함수의 개념과 종류

● SQL 함수

- 데이터를 조회하거나 집계, 저장, 수정하는 과정에서 값을 가공하기 위해 제공되는 모듈화된 기능
- SQL 함수는 DBMS에 미리 만들어져 저장되어 있는지, 사용자가 필요에 따라서 직접 만드는지에 따라 내장 함수와 사용자 정의 함수로 구분함



Section 02

단일 행 함수의 종류와 사용법

문자형 함수

- 문자형 함수

- 문자열을 입력하면 문자열이나 숫자 값을 반환하는 함수

- **CHAR_LENGTH(), LENGTH()**

- CHAR_LENGTH() : 문자의 개수를 반환하는 함수
- LENGTH() : 문자열에 할당된 바이트(Byte) 수를 반환하는 함수
- 형식

```
CHAR_LENGTH(문자열)
```

```
LENGTH(문자열)
```

문자형 함수

- [예제 3-1] 영문 'HELLO'와 한글 '안녕'의 문자 개수를 세고 결과를 확인하세요.

▶ 실행결과

CHAR_LENGTH('HELLO')	LENGTH('HELLO')	CHAR_LENGTH('안녕')	LENGTH('안녕')
5	5	2	6

문자형 함수

● CONCAT(), CONCAT_WS()

- CONCAT() : 문자열을 연결할 때 사용하는 함수
- CONCAT_WS() : 구분자와 함께 문자열을 연결할 때 사용하는 함수
- 형식

CONCAT(문자열1, 문자열2, ...)

CONCAT_WS(구분자, 문자열1, 문자열2, ...)

- [예제 3-2] CONCAT() 함수와 CONCAT_WS() 함수를 사용하여 문자열을 연결했을 때의 결과값을 비교하세요.

▶ 실행결과

CONCAT('DREAMS', 'COME', 'TRUE')	CONCAT_WS('-', '2023', '01', '29')
DREAMSCOMETRUE	2023-01-29

문자형 함수

● LEFT(), RIGHT(), SUBSTR()

- LEFT() : 문자열의 왼쪽부터 길이만큼 문자열을 반환하는 함수
- RIGHT() : 문자열의 오른쪽부터 길이만큼 문자열을 반환하는 함수
- SUBSTR() : 지정한 위치로부터 길이만큼의 문자열을 반환하는 함수
- 형식

LEFT(문자열, 길이)

RIGHT(문자열, 길이)

SUBSTR(문자열, 시작_위치, 길이) 또는 SUBSTRING(문자열, 시작_위치, 길이)

문자형 함수

- [예제 3-3] '문자형 처리 함수' 문자열에서 지정한 길이만큼 문자열을 반환하세요.

문자형 함수

● SUBSTRING_INDEX()

- SUBSTRING_INDEX() : 지정한 구분자를 기준으로 문자열을 분리해서 가져올 때 사용하는 함수
- 형식

```
SUBSTRING_INDEX(문자열, 구분자, 인덱스)
```

- [예제 3-4] '서울시 동작구 흑석로'에서 앞/뒤 2번째 위치에 있는 공백(' ')을 기준으로 문자열을 분리한 결과를 확인하세요.

▶ 실행결과

SUBSTRING_INDEX('서울시 동작구 흑석로', ' ', 2)	SUBSTRING_INDEX('서울시 동작구 흑석로', ' ', -2)
서울시 동작구	동작구 흑석로

문자형 함수

● LPAD(), RPAD()

- LPAD()와 RPAD() : 지정한 길이에서 문자열을 제외한 빈칸을 특정 문자로 채울 때 사용하는 함수
- LPAD()는 왼쪽에, RPAD()는 오른쪽에 특정 문자를 채움
- 형식

LPAD(문자열, 길이, 채울_문자열)

RPAD(문자열, 길이, 채울_문자열)

- [예제 3-5] 'SQL' 문자열의 앞 7칸에 # 기호를 채우고, SQL 뒤 2칸에는 * 기호를 채우세요.

▶ 실행결과

LPAD('SQL', 10, '#')	RPAD('SQL', 5, '*')
#####SQL	SQL**

문자형 함수

● LTRIM(), RTRIM()

- LTRIM() : 왼쪽의 공백을 제거할 때 사용하는 함수
- RTRIM() : 오른쪽 공백을 제거할 때 사용하는 함수
- 형식

LTRIM(문자열)

RTRIM(문자열)

- [예제 3-6] ' SQL ' 문자열의 앞/뒤에 있는 공백을 제거하고, 결과를 문자열의 길이로 확인하세요.

▶ 실행결과

LENGTH(LTRIM(' SQL '))	LENGTH(RTRIM(' SQL '))	LENGTH(TRIM(' SQL '))
4	4	3

문자형 함수

● TRIM(BOTH/LEADING/TRAILING)

- 양쪽에 있는 동일 문자열을 제거하고자 할 때에는 TRIM()의 첫 번째 매개변수에 BOTH를 넣음
- BOTH 대신 LEADING을 넣으면 왼쪽에 있는 문자열이, TRAILING을 넣으면 오른쪽에 있는 문자열이 제거됨
- 형식

TRIM(문자열)

TRIM(제거할_문자열의_방향(BOTH/LEADING/TRAILING) 제거할_문자열 FROM 문자열)

- [예제 3-7] 'abcSQLabcabc'에서 BOTH/LEADING/TRAILING을 사용하여 앞뒤에 있는 'abc' 문자열을 제거하세요.

▶ 실행결과

TRIM(BOTH '*' FROM '**SQL**')	TRIM(LEADING 'abc' FROM 'abcSQLabc')	TRIM(TRAILING 'abc' FROM 'abcSQLabcabc')
SQL	SQLabc	abcSQL

문자형 함수

● FIELD(), FIND_IN_SET(), INSTR(), LOCATE()

- FIELD() : 여러 문자열 중에서 찾는 문자열이 있으면 문자열의 위치 값을 반환하는 함수. 만약 찾는 문자열이 없으면 0을 반환함

- 형식

```
FIELD(찾을_문자열, 문자열1, 문자열2, ...)
```

- FIND_IN_SET() : 문자열 리스트에서 지정한 문자열을 찾아서 위치 값을 반환함

- 형식

```
FIND_IN_SET(찾을_문자열, 문자열_리스트)
```

- INSTR(), LOCATE() : 기준 문자열 중에서 부분 문자열을 찾아서 위치 값을 반환함

- 형식

```
INSTR(기준_문자열, 부분_문자열)
```

```
LOCATE(부분_문자열, 기준_문자열)
```

문자형 함수

- [예제 3-8] 문자열의 위치를 찾는 4가지 함수 FIELD(), FIND_IN_SET(), INSTR(), LOCATE()를 사용하고 결과를 확인하세요.

▶ 실행결과

FIELD('JAVA', 'SQL', 'JAVA', 'C')	FIND_IN_SET('JAVA', 'SQL,JAVA,C')	INSTR('네 인생을 살아라', '인생')	LOCATE('인생', '네 인생을 살아라')
2	2	3	3

문자형 함수

● ELT()

- ELT() : 지정한 위치에 있는 문자열을 반환하는 함수
- FIELD()는 첫 매개변수 값에 찾고자 하는 문자열을 넣어주지만, ELT()는 찾을 문자열의 위치 값을 넣음
- 형식

```
ELT(찾을_문자열_위치, 문자열1, 문자열2, ...)
```

- [예제 3-9] 문자열의 위치를 찾는 4가지 함수 FIELD(), FIND_IN_SET(), INSTR(), LOCATE()를 사용하고 결과를 확인하세요.

▶ 실행결과

```
ELT(2, 'SQL', 'JAVA', 'C')
```

```
JAVA
```

문자형 함수

● REPEAT()

- REPEAT() : 문자열을 반복하고자 할 때 사용하는 함수
 - 매개변수에 반복할 문자열과 반복할 횟수를 넣음
- 형식

```
REPEAT(문자열, 횟수)
```

- [예제 3-10] REPEAT()를 사용하여 '*' 5개를 반복하세요.

▶ 실행결과

```
REPEAT('*', 5)
```

```
*****
```

문자형 함수

● REPLACE()

- REPLACE() : 문자열의 일부를 다른 문자열로 대체하고자 할 때 사용하는 함수
- 형식

```
REPLACE(문자열, 원래_문자열, 바꿀_문자열)
```

- [예제 3-11] '.'로 구분되어 있는 전화번호의 구분자를 '-'로 대체하세요.

▶ 실행결과

REPLACE('010.1234.5678', '.', '-')
010-1234-5678

문자형 함수

● REVERSE()

- REVERSE() : 문자열을 거꾸로 뒤집을 때 사용하는 함수
- 형식

```
REVERSE(문자열)
```

- [예제 3-12] 'OLLEH'를 맨 뒤쪽의 문자부터 나타내세요.

▶ 실행결과

```
REVERSE('OLLEH')
```

```
HELLO
```

숫자형 함수

- 숫자형 함수

- 숫자를 다루는 단일 행 함수를 통해 올림, 버림, 반올림, 절댓값 반환, 나머지, 제곱승 등의 기능을 수행할 수 있음

- **CEILING(), FLOOR(), ROUND(), TRUNCATE()**

- CEILING() : 올림, FLOOR() : 버림, ROUND() : 지정한 위치에서 반올림, TRUNCATE() : 지정한 위치에서 버림 작업을 수행하는 함수
- 형식

CEILING(숫자)

FLOOR(숫자)

ROUND(숫자)

ROUND(숫자, 반올림할_자릿수)

TRUNCATE(숫자, 버림할_자릿수)

숫자형 함수

- [예제 3-13] 123.56에 대해 CEILING(), FLOOR(), ROUND(), TRUNCATE()를 사용한 결과를 확인하세요.

▶ 실행결과

CEILING(123.56)	FLOOR(123.56)	ROUND(123.56)	ROUND(123.56, 1)	TRUNCATE(123.56, 1)
124	123	124	123.6	123.5

숫자형 함수

● ABS(), SIGN()

- ABS() : 절댓값을 반환하는 함수
- SIGN() : 양수의 경우 1, 음수의 경우 -1을 반환하는 함수
- 형식

ABS(숫자)
SIGN(숫자)

- [예제 3-14] -120과 120의 절댓값과 음수, 양수 여부를 판별하세요.

▶ 실행결과

ABS(-120)	ABS(120)	SIGN(-120)	SIGN(120)
120	120	-1	1

숫자형 함수

● MOD()

- MOD() : 나머지를 구하는 함수로 세 가지 방법을 사용할 수 있음
- 형식

MOD(숫자1, 숫자2)

숫자1 % 숫자2

숫자1 MOD 숫자2

- [예제 3-15] 203을 4로 나눈 나머지를 확인하세요.

▶ 실행결과

MOD(203, 4)	203 % 4	203 MOD 4
3	3	3

숫자형 함수

● POWER(), SQRT(), RAND()

- POWER() : n제곱승 값을 반환하는 함수
- SQRT() : 제곱근 값을 반환하는 함수
- RAND() : 0과 1사이 임의의 실수 값을 반환하는 함수
 - 매개변수 값을 넣지 않으면 실행할 때마다 0과 1 사이에 있는 임의의 실수를 반환함
 - RAND() 안에 시드(Seed)를 설정하면 매번 동일한 임의의 값을 얻을 수 있음
 - ROUND(), TRUNCATE() 등의 함수를 같이 사용하면 임의의 정수를 만들 수 있음
- 형식

POWER(숫자1, 숫자2) ● ————— 숫자1의 숫자2 제곱승

SQRT(숫자)

RAND()

RAND(숫자)

숫자형 함수

- [예제 3-16] 제곱과 제곱근, 랜덤값을 구하는 함수를 사용하고, 그 결과를 보이세요.

▶ 실행결과

POWER(2, 3)	SQRT(16)	RAND()	RAND(100)	ROUND(RAND() * 100)
8	4	0.838829533046884	0.17353134804734155	76

날짜/시간형 함수

- 현재 날짜/시간 반환 함수

- NOW(), SYSDATE(), CURDATE(), CURTIME()
 - 현재 날짜와 시간을 반환하는 함수
- **NOW(), SYSDATE()** : 시스템의 현재 날짜와 시간을 반환함
- **CURDATE()** : 시스템의 현재 날짜를 반환함
- **CURTIME()** : 시스템의 현재 시간을 반환함

- [예제 3-17] 현재 날짜와 시간을 다양한 형태로 반환하세요.

▶ 실행결과

NOW()	SYSDATE()	CURDATE()	CURTIME()
2023-01-31 14:31:07	2023-01-31 14:31:07	2023-01-31	14:31:07

날짜/시간형 함수

- 연도, 분기, 월, 일, 시, 분, 초 반환 함수

- YEAR(), QUARTER(), MONTH(), DAY(), HOUR(), MINUTE(), SECOND()

- 날짜 중 해당 값을 반환함

- [예] YEAR()는 날짜 중 연도를, QUARTER()는 날짜 중 분기를 반환함

- [예제 3-18] 날짜에 관한 다양한 함수의 결과를 확인하세요.

▶ 실행결과

NOW()	YEAR(NOW())	QUARTER(NOW())	MONTH(NOW())	DAY(NOW())	HOUR(NOW())	MINUTE(NOW())	SECOND(NOW())
2023-01-29 20:23:32	2023	1	1	29	20	23	32

날짜/시간형 함수

● 기간 반환 함수

- DATEDIFF(), TIMESTAMPDIFF()
 - 지정한 기간을 반환하는 함수
- **DATEDIFF()** : 기간을 일자 기준으로 반환함
- **TIMESTAMPDIFF()** : 기간을 지정한 단위 기준으로 보여줌
 - DATEDIFF()는 끝 일자를 앞에 넣어주고, TIMESTAMPDIFF()는 끝 일자를 뒤에 넣음
- 형식

DATEDIFF(끝_일자, 시작_일자)

TIMESTAMPDIFF(단위, 시작_일자, 끝_일자)

TIMESTAMPDIFF() 함수에 매개변수로 들어가는 단위

단위	의미	단위	의미
SECOND	초	WEEK	주
MINUTE	분	MONTH	월
hour	시	QUARTER	분기
DAY	일	YEAR	연도

날짜/시간형 함수

- [예제 3-19] 현재 날짜를 기준으로 날짜 사이의 기간을 확인하세요.

▶ 실행결과

NOW()	DATEDIFF('2025-12-20', NOW())	DATEDIFF(NOW(), '2025-12-20')	TIMESTAMPDIFF(YEAR, NOW(), '2025-12-20')	TIMESTAMPDIFF(MONTH, NOW(), '2025-12-20')	TIMESTAMPDIFF(DAY, NOW(), '2025-12-20')
2023-01-31 14:35:11	1054	-1054	2	34	1053

날짜/시간형 함수

- 기간을 반영하는 날짜 함수

- **ADDDATE()** : 지정한 날짜를 기준으로 그 기간만큼 더한 날짜를 반환하는 함수
- **SUBDATE()** : 기간만큼 뺀 날짜를 반환함
- 형식

ADDDATE(날짜, 기간)

또는 ADDDATE(날짜, INTERVAL 기간 단위)

SUBDATE(날짜, 기간)

또는 SUBDATE(날짜, INTERVAL 기간 단위)

- [예제 3-20] 오늘 날짜 및 오늘 날짜로부터 50일 후, 50개월 후, 50시간 전의 날짜를 확인하세요.

▶ 실행결과

NOW()	ADDDATE(NOW(), 50)	ADDDATE(NOW(), INTERVAL 50 DAY)	ADDDATE(NOW(), INTERVAL 50 MONTH)	SUBDATE(NOW(), INTERVAL 50 HOUR)
2023-01-31 14:59:41	2023-03-22 14:59:41	2023-03-22 14:59:41	2027-03-31 14:59:41	2023-01-29 12:59:41

날짜/시간형 함수

- 기타 날짜 반환 함수

- **LAST_DAY()** : 해당 월의 마지막 일자를 반환함
- **DAYOFYEAR()** : 현재 연도에서 며칠이 지났는지를 반환함
- **MONTHNAME()**은 월을 영문으로, **WEEKDAY()**는 요일을 정수로 보여줌
- 형식

LAST_DAY(날짜)

DAYOFYEAR(날짜)

MONTHNAME(날짜)

WEEKDAY(날짜)

날짜/시간형 함수

- [예제 3-21] 오늘 날짜를 기준으로 이번 달 마지막 날짜와 일 년 중 몇 일째인지, 그리고 월 이름과 요일을 확인하세요.

▶ 실행결과

NOW()	LAST_DAY(NOW())	DAYOFYEAR(NOW())	MONTHNAME(NOW())	WEEKDAY(NOW())
2023-01-31 15:09:37	2023-01-31	31	January	1

Section 03

기타 단일 행 함수

형 변환 함수

- 형 변환 함수

- 데이터를 검색, 삽입할 때 컬럼에 맞는 형식으로 지정하지 않으면 오류가 나는 경우에 사용하는 함수

- **CAST(), CONVERT()**

- 원하는 형태로 데이터타입을 변경하여 처리하거나 확인할 수 있음
- 데이터 타입: CHAR, SIGNED(부호 있는 숫자), UNSIGNED(부호 없는 숫자), DATE, DATETIME, TIME, BINARY 등을 넣을 수 있음
- 형식

CAST(값 AS 데이터타입)

CONVERT(값, 데이터타입)

형 변환 함수

- [예제 3-22] 문자 '1'을 부호 없는 숫자 형식으로, 숫자 2를 문자 형식으로 변환하세요.

▶ 실행결과

CAST('1' AS UNSIGNED)	CAST(2 AS CHAR(1))	CONVERT('1', UNSIGNED)	CONVERT(2, CHAR(1))
1	2	1	2

제어 흐름 함수

● 제어 흐름 함수

- 프로그램의 흐름을 제어할 때 사용함
- 조건에 따른 결과를 SQL 문장 하나로 얻을 수 있음

● IF

- IF(조건, 수식1, 수식2) : 조건의 결과가 참이면 수식1을 반환하고, 그렇지 않으면 수식2의 결과를 반환함
- 형식

```
IF(조건, 수식1, 수식2)
```

- [예제 3-23] 12,500원짜리 제품을 450개 이상 주문한 금액이 5,000,000원 이상이면 '초과달성', 미만이면 '미달성'이라고 보이세요.

▶ 실행결과

```
IF(12500 * 450 > 5000000, '초과달성', '미달성')
```

```
초과달성
```

제어 흐름 함수

● IFNULL(), NULLIF()

- NULL과 관련된 제어 흐름 함수
- IFNULL() : 수식1이 NULL이 아니면 수식1의 값을 반환하고, NULL이면 수식2의 값을 반환함
- NULLIF() : 두 수식 값을 비교하여 값이 같으면 NULL을 반환하고, 값이 다르면 수식1의 값을 반환함
- 형식

```
IFNULL(수식1, 수식2)
```

```
NULLIF(수식1, 수식2)
```

제어 흐름 함수

- [예제 3-24] IFNULL()의 첫 매개변수 값이 NULL인지 여부에 따라 어떻게 결과가 다르게 나오는지 확인하세요.

▶ 실행결과

IFNULL(1, 0)	IFNULL(NULL, 0)	IFNULL(1/0, 'OK')
1	0	OK

- [예제 3-25] NULLIF()을 사용하여 두 결과를 비교하세요.

▶ 실행결과

NULLIF(12 * 10, 120)	NULLIF(12 * 10, 1200)
NULL	120

CASE문

● CASE문

- 함수는 아니지만 조건 비교가 여러 개일 때 유용하게 사용할 수 있음
- WHEN 조건1 THEN 값1 WHEN 조건2 THEN 값2...
 - 모든 조건을 만족하지 않으면 ELSE 다음에 값을 넣어줌
 - CASE문은 END로 마무리되어야 함

- [예제 3-26] 주문 금액이 5,000,000원 이상이면 '초과달성', 4,000,000원 이상이면 '달성' 그 나머지는 '미달성'이라고 할 때, 12,500원짜리 제품을 450개 이상 주문했다면 어디에 해당하는지 보이세요.

▶ 실행결과

```
CASE WHEN 12500 * 450 > 5000000 THEN '초과달성'  
      WHEN 2500 * 450 > 4000000 THEN '달성'  
      ELSE '미달성' END
```

초과달성